



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Laboratorio S11

Persistencia políglota y caché

MATERIA

Sistemas Distribuidos

Alumno

Reyes Vázquez Fernando

PROFESOR

Oscar Manuel Ruiz Hurtado

FECHA DEL REPORTE | 12 DE NOVIEMBRE DE 2026

GRUPO 02 | SEMESTRE 2027-1

1. El punto de partida

Preparación del entorno

Para comenzar trabajé en una copia de Mexi Banco S11 dentro de mi carpeta SD_Clase. Levanté los servicios con Docker Compose y verifiqué que PostgreSQL, Redis, RabbitMQ y MongoDB respondieran. Durante el arranque apareció un problema con MongoDB 8 y el kernel de Docker, así que ajusté la configuración del contenedor y comprobé que finalmente quedara saludable.

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % nano docker-compose.yml
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose config --quiet
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose up -d mongo
[+] up 1/1
 ✔ Container mexi-banco-s11-mongo-1 Started                                                
```

registraba aciertos ni fallos, porque la aplicación no estaba usando la caché. Aunque el saldo ya era $800.00, la fecha de lectura cambió con cada respuesta.

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs cuenta | grep -c "leida de Postgres"
4
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % for i in $(seq 1 10); do
  curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
done
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T02:49:11.281209844Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T02:49:11.524264094Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T02:49:11.783623635Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T02:49:11.889032385Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T02:49:11.943076219Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T02:49:11.968847802Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T02:49:11.993597261Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T02:49:12.025180302Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T02:49:12.048918219Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T02:49:12.073339386Z"}
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs cuenta | grep -c "leida de Postgres"
14
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:0
keyspace_misses:0
```

Figura 5. *Diez consultas consecutivas y aumento de 4 a 14 lecturas a PostgreSQL.*

2. La caché del saldo en Redis

Implementación de la caché

Configuré el servicio cuenta con el patrón cache-aside. Agregué @EnableCaching y un administrador de caché Redis para la colección cuentas. Establecí un TTL de 30 segundos y un serializador JSON para CuentaVista, de modo que Redis guardara datos comprensibles en lugar de la serialización binaria de Java.

Coloqué @Cacheable en ConsultaDeSaldo.consultar(), porque ese método es el que atiende lo que ve el cliente. Cuando no existe la clave, consulta PostgreSQL y guarda el resultado; si ya existe, responde con Redis. En cambio, CuentaService sigue leyendo PostgreSQL para decidir si puede autorizar un cargo. Así, la caché no se vuelve la fuente de verdad del dinero.

```
diff --git a/cuenta/src/main/java/mx/mexibanco/cuenta/ConfiguracionCache.java b/cuenta/src/main/java/mx/mexibanco/cuenta/ConfiguracionCache.java
index 7aa4a84..0072775 100644
--- a/cuenta/src/main/java/mx/mexibanco/cuenta/ConfiguracionCache.java
+++ b/cuenta/src/main/java/mx/mexibanco/cuenta/ConfiguracionCache.java
@@ -1,6 +1,17 @@
+
 package mx.mexibanco.cuenta;

+import com.fasterxml.jackson.databind.ObjectMapper;
+import org.springframework.beans.factory.annotation.Value;
+import org.springframework.boot.autoconfigure.cache.RedisCacheManagerBuilderCustomizer;
+import org.springframework.cache.annotation.EnableCaching;
+import org.springframework.context.annotation.Bean;
 import org.springframework.context.annotation.Configuration;
+import org.springframework.data.redis.cache.RedisCacheConfiguration;
+import org.springframework.data.redis.serializer.Jackson2JsonRedisSerializer;
+import org.springframework.data.redis.serializer.RedisSerializationContext.SerializationPair;
+
+import java.time.Duration;

 /**
  * v11: la cache de GET /cuentas/{clabe} en Redis, con el patron cache-aside.
@@ -15,12 +26,22 @@ import org.springframework.context.annotation.Configuration;
  * Lo que ya esta listo: el host y el puerto de Redis (spring.data.redis en application.yml) y
  * spring.cache.type, que con CACHE_TIPO=none apaga la cache sin tocar codigo.
  */
-// TODO S11: @EnableCaching, sin el, @Cacheable y @CacheEvict se ignoran sin aviser.
+@EnableCaching
 @Configuration
 public class ConfiguracionCache {

+       /** El nombre de la cache. En Redis cada entrada queda como la clave "cuentas::<clabe>". */
+       public static final String CUENTAS = "cuentas";
+
-       // TODO S11: un @Bean RedisCacheManagerBuilderCustomizer que a la cache CUENTAS le ponga entryTtl(ttl), valores con Jackson2JsonRedisSerializer<>(json, CuentaVista.class) y llame a transactionAw
are().
+       @Bean
+       RedisCacheManagerBuilderCustomizer cacheDeCuentas(
+                       @Value("${mexibanco.cache.ttl:30s}") Duration ttl, ObjectMapper json) {
+               RedisCacheConfiguration cuentas = RedisCacheConfiguration.defaultCacheConfig()
+                       .entryTtl(ttl)
+                       .disableCachingNullValues()
+                       .serializeValuesWith(SerializationPair.fromSerializer(
+                               new Jackson2JsonRedisSerializer<>(json, CuentaVista.class)));
+               return builder -> builder.withCacheConfiguration(CUENTAS, cuentas)
+                       .transactionAware();
+       }
 }
diff --git a/cuenta/src/main/java/mx/mexibanco/cuenta/ConsultaDeSaldo.java b/cuenta/src/main/java/mx/mexibanco/cuenta/ConsultaDeSaldo.java
index 5558ldc..38c1001 100644
--- a/cuenta/src/main/java/mx/mexibanco/cuenta/ConsultaDeSaldo.java
+++ b/cuenta/src/main/java/mx/mexibanco/cuenta/ConsultaDeSaldo.java
@@ -4,6 +4,7 @@ import mx.mexibanco.compartido.NoEncontradoException;
 import org.slf4j.Logger;
 import org.slf4j.LoggerFactory;
 import org.springframework.stereotype.Service;
+import org.springframework.cache.annotation.Cacheable;

 /**
  * v11: la consulta que el cliente hace para VER su saldo (GET /cuentas/{clabe}). Es la unica que se
@@ -26,6 +27,7 @@ public class ConsultaDeSaldo {
        }
:
```

Figura 6. *Cambios de ConfiguracionCache.java y configuración de la caché con Redis.*



Figura 7. *Anotaciones @Cacheable y @CacheEvict utilizadas en el servicio cuenta.*

Para los métodos cargar y abonar agregué @CacheEvict, de forma que se elimine la copia de la cuenta cuando cambia su saldo. La configuración transactionAware() hace que esa invalidación ocurra después de confirmar la transacción local, evitando borrar una copia por una operación que después pudiera revertirse.

Tres instancias compartiendo la misma copia

Reconstruí el servicio cuenta y lo levanté con tres instancias. Realicé seis consultas al saldo de Ana y las respuestas pasaron por distintas copias, algo visible en el encabezado X-Instancia. Aun así, todas devolvieron la misma fecha leidaEn: la información se estaba compartiendo desde Redis y no se consultaba PostgreSQL en cada petición.

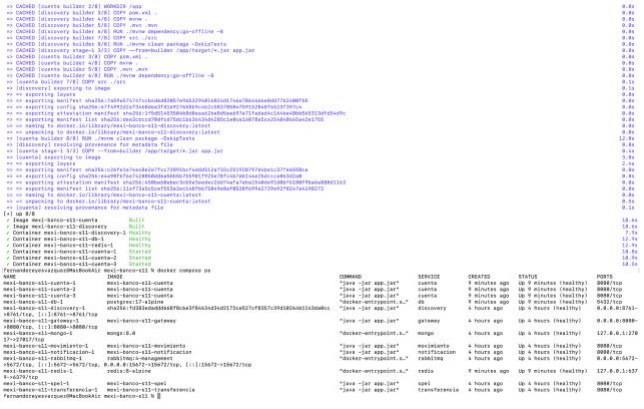


Figura 8. *Reconstrucción del servicio cuenta con tres instancias activas.*

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % for i in 1 2 3 4 5 6; do
  curl -s -D - http://localhost:8080/api/cuentas/002180000000000001 | grep -iE 'x-instancia|leidaEn'
done
X-Instancia: a2c5eeb69e6c
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T06:33:44.899264634Z"}
X-Instancia: 1b1f430c569a
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T06:33:44.899264634Z"}
X-Instancia: 11987f2ed451
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T06:33:44.899264634Z"}
X-Instancia: 1b1f430c569a
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T06:33:44.899264634Z"}
X-Instancia: a2c5eeb69e6c
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T06:33:44.899264634Z"}
X-Instancia: 11987f2ed451
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T06:33:44.899264634Z"}
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs --since 30s cuenta | grep -c "leida de Postgres"
1
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T redis redis-cli GET cuentas::002180000000000001

fernandoreyesvazquez@MacBookAir mexi-banco-s11 % curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T06:35:48.978746636Z"}
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T redis redis-cli GET cuentas::002180000000000001
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T06:35:48.978746636Z"}
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
9
```

Figura 9. *Respuestas de diferentes instancias con la misma fecha de lectura y verificación del valor JSON y TTL.*

También consulté directamente la clave cuentas::002180000000000001 en Redis y observé el objeto JSON, además de su TTL. Cuando realicé una nueva transferencia, el monitor de Redis mostró la operación DEL sobre las claves de las cuentas involucradas. Así comprobé que la copia se invalida y que el siguiente GET vuelve a traer un saldo actualizado de PostgreSQL.

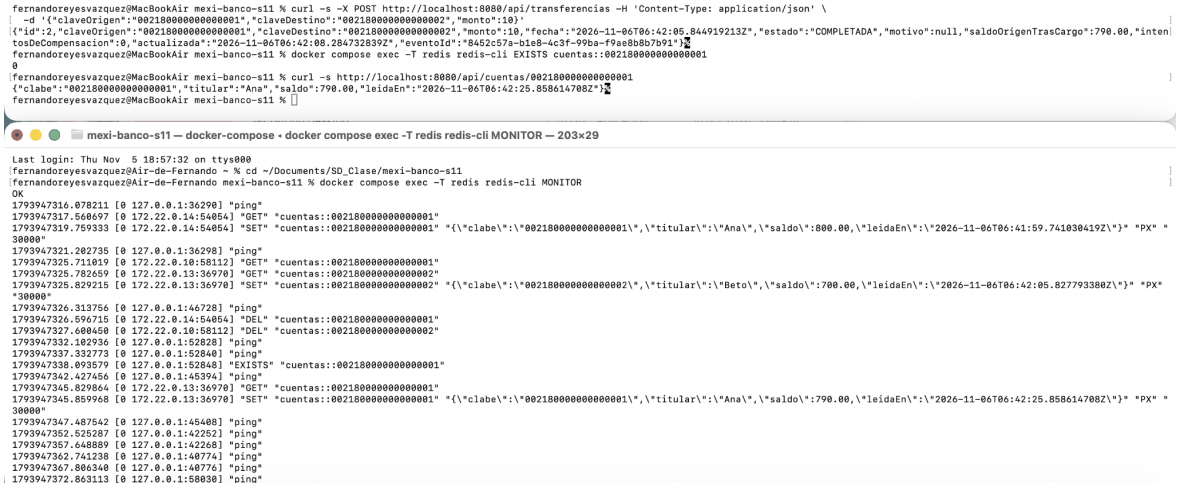


Figura 10. *Redis MONITOR: claves consultadas y eliminadas después de una transferencia.*

3. Mediciones con y sin caché

Hit ratio: dos escenarios

Primero reinicié los contadores de Redis con CONFIG RESETSTAT. En el escenario A hice cien consultas consecutivas: registré 99 aciertos y un fallo, que equivalen a un hit ratio del 99 %. La primera consulta llenó la caché y las siguientes aprovecharon la misma copia.

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T redis redis-cli CONFIG RESETSTAT
OK
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
done
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:99
keyspace_misses:1
```

Figura 11. *Escenario A: 99 aciertos y un fallo en cien consultas.*

En el escenario B intercalé una transferencia de $1 cada diez consultas. Se registraron 100 aciertos y 20 fallos, es decir, un hit ratio de 83.33 %. Bajó porque las transferencias invalidaban las claves y las siguientes consultas debían volver a obtener el saldo desde PostgreSQL.

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T redis redis-cli CONFIG RESETSTAT
OK
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001

  if [ $((i % 10)) -eq 0 ]; then
    curl -s -X POST http://localhost:8080/api/transferencias \
      -H 'Content-Type: application/json' \
      -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1}'
    echo
  fi
done
{"id":3,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1,"fecha":"2026-11-06T06:53:18.014509677Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":789.00,"inten
tosDeCompensacion":0,"actualizada":"2026-11-06T06:53:18.800225011Z","eventoId":"ba2ae27a-77ce-46c6-919c-b20647cd4bdc"}
{"id":4,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1,"fecha":"2026-11-06T06:53:19.405990094Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":788.00,"inten
tosDeCompensacion":0,"actualizada":"2026-11-06T06:53:22.789854679Z","eventoId":"78d78773-f328-4108-bbc8-8200ee1cbb54"}
{"id":5,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1,"fecha":"2026-11-06T06:53:23.300113138Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":787.00,"inten
tosDeCompensacion":0,"actualizada":"2026-11-06T06:53:23.431801680Z","eventoId":"a8833bd2-c680-4535-87eb-9c3a4cea12a5"}
{"id":6,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1,"fecha":"2026-11-06T06:53:23.689641596Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":786.00,"inten
tosDeCompensacion":0,"actualizada":"2026-11-06T06:53:23.833010472Z","eventoId":"2aab4f2e-247e-4664-9c21-334a70f37d2e"}
{"id":7,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1,"fecha":"2026-11-06T06:53:24.086432263Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":785.00,"inten
tosDeCompensacion":0,"actualizada":"2026-11-06T06:53:24.163529138Z","eventoId":"c3a97869-e4e8-4c17-a34a-dd0f4fe391ba"}
{"id":8,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1,"fecha":"2026-11-06T06:53:24.391250222Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":784.00,"inten
tosDeCompensacion":0,"actualizada":"2026-11-06T06:53:24.477726764Z","eventoId":"f6772b5b-1630-432a-be16-5dccc6a208d5"}
{"id":9,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1,"fecha":"2026-11-06T06:53:24.710741347Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":783.00,"inten
tosDeCompensacion":0,"actualizada":"2026-11-06T06:53:24.794818055Z","eventoId":"ce6f90a7-1bba-4355-a19a-e411fefb03be"}
{"id":10,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1,"fecha":"2026-11-06T06:53:25.046562389Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":782.00,"inte
ntosDeCompensacion":0,"actualizada":"2026-11-06T06:53:25.176206333Z","eventoId":"e658ac20-185f-4e86-9cc3-ee285ec23894"}
{"id":11,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1,"fecha":"2026-11-06T06:53:25.494991833Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":781.00,"inte
ntosDeCompensacion":0,"actualizada":"2026-11-06T06:53:25.555687875Z","eventoId":"fc6578e7-b55e-471b-abc8-cccfebd5df9d"}
{"id":12,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1,"fecha":"2026-11-06T06:53:25.790015500Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":780.00,"inte
ntosDeCompensacion":0,"actualizada":"2026-11-06T06:53:25.878021792Z","eventoId":"fc66650d-4b3c-498e-b5e0-8932a0867f3a"}
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:100
keyspace_misses:20
```

Figura 12. *Escenario B: transferencias intercaladas, 100 hits y 20 misses.*

Escenario	Aciertos	Fallos	Hit ratio
A: consultas repetidas	99	1	99 %
B: con transferencias	100	20	83.33 %

Latencia y lecturas a PostgreSQL

Medí el tiempo de cien consultas en dos corridas con Redis y en otras dos con la caché desactivada mediante CACHE_TIPO=none. Registré promedio, mediana, p95 y cuántas lecturas nuevas aparecían en los logs del servicio cuenta. Al terminar volví a activar Redis para dejar el proyecto como estaba.

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs cuenta | grep -c "leida de Postgres"
26
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % for i in $(seq 1 100); do
  curl -s -o /dev/null -w '%{time_total}\n' \
    http://localhost:8080/api/cuentas/002180000000000001
done | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000}
END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=14.37 ms mediana=8.40 ms p95=28.48 ms
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs cuenta | grep -c "leida de Postgres"
27
```

Figura 13. *Primera corrida con caché: promedio 14.37 ms y una lectura nueva a PostgreSQL.*

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs cuenta | grep -c "leida de Postgres"
27
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % for i in $(seq 1 100); do
  curl -s -o /dev/null -w '%{time_total}\n' \
    http://localhost:8080/api/cuentas/002180000000000001
done | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000}
END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=15.87 ms mediana=11.46 ms p95=28.80 ms
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs cuenta | grep -c "leida de Postgres"
28
```

Figura 14. *Segunda corrida con caché: promedio 15.87 ms y una lectura nueva.*

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T cuenta printenv CACHE_TIPO
none
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs cuenta | grep -c "leida de Postgres"
0
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % for i in $(seq 1 100); do
  curl -s -o /dev/null -w '%{time_total}\n' \
    http://localhost:8080/api/cuentas/002180000000000001
done | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000}
END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=27.36 ms mediana=9.65 ms p95=32.05 ms
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs cuenta | grep -c "leida de Postgres"
100
```

Figura 15. *Primera corrida sin caché: promedio 27.36 ms y cien lecturas nuevas.*

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs cuenta | grep -c "leida de Postgres"
100
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % for i in $(seq 1 100); do
  curl -s -o /dev/null -w '%{time_total}\n' \
    http://localhost:8080/api/cuentas/002180000000000001
done | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000}
END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=16.68 ms mediana=11.17 ms p95=51.02 ms
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs cuenta | grep -c "leida de Postgres"
200
```

Figura 16. *Segunda corrida sin caché: promedio 16.68 ms y cien lecturas nuevas.*

Métrica	Con caché 1	Con caché 2	Sin caché 1	Sin caché 2
Promedio (ms)	14.37	15.87	27.36	16.68
Mediana (ms)	8.40	11.46	9.65	11.17
p95 (ms)	28.48	28.80	32.05	51.02
Lecturas a PostgreSQL	1	1	100	100

Con caché hubo solo una lectura nueva a PostgreSQL por corrida, frente a cien lecturas sin caché. Los promedios con Redis fueron menores en estas pruebas, pero la latencia varió entre corridas e incluso algunas medidas se acercaron. El resultado más claro fue que Redis redujo la carga sobre PostgreSQL y permitió atender muchas más consultas reutilizando los datos temporales.

4. Los avisos en MongoDB

Documentos y registro sin duplicar

En la segunda parte moví los avisos a la colección avisos de MongoDB. En BitacoraDeAvisos.documento() construí un documento por destinatario con _id formado por eventoId y CLABE, además de monto y saldo como Decimal128 y fecha como Date. Los campos específicos cambian según el canal: SMS guarda teléfono y texto; CORREO guarda dirección, asunto y cuerpo; PUSH guarda token, título y cuerpo.

```
92      public static Document documento(EventoTransferencia evento, Destinatario destinatario, Contacto contacto) {
93          Document aviso = new Document("_id", evento.eventoId() + ":" + destinatario.clabe())
94              .append("eventoId", evento.eventoId().toString())
95              .append("transferenciaId", evento.transferenciaId())
96              .append("clabe", destinatario.clabe())
97              .append("tipo", destinatario.tipo())
98              .append("monto", new Decimal128(evento.monto()))
99              .append("saldo", new Decimal128(destinatario.saldo()))
100             .append("fecha", Date.from(evento.fecha()));
101
102         String texto = texto(destinatario, evento);
103
104         switch (contacto) {
105             case Contacto.Sms sms -> aviso
106                 .append("canal", "SMS")
107                 .append("telefono", sms.telefono())
108                 .append("texto", texto);
109
110             case Contacto.Correo correo -> aviso
111                 .append("canal", "CORREO")
112                 .append("correo", correo.correo())
113                 .append("asunto", destinatario.tipo())
114                 .append("cuerpo", texto);
115
116             case Contacto.Push push -> aviso
117                 .append("canal", "PUSH")
118                 .append("token", push.token())
119                 .append("titulo", destinatario.tipo())
120                 .append("cuerpo", texto);
121         }
122
123         return aviso;
124     }
```

Figura 17. *Método documento(): campos comunes y estructuras distintas para SMS, CORREO y PUSH.*

En registrar(evento) inserté los avisos uno por uno. Si ya existe el _id, se captura DuplicateKeyException y no se cuenta como un aviso nuevo. Es importante hacerlo individualmente: si una ejecución anterior guardó solamente el primer aviso, una reentrega puede completar el segundo sin duplicar el primero.

```
84      public int registrar(EventoTransferencia evento) {
85          int nuevos = 0;
86          for (Destinatario destinatario : destinatarios(evento)) {
87              Document aviso = documento(evento, destinatario, contactos.de(destinatario.clabe()));
88              try {
89                  mongo.insert(aviso, COLECCION);
90                  nuevos++;
91              } catch (DuplicateKeyException yaEstaba) {
92                  log.info("[avisos] {} ya estaba: evento repetido, no se duplica", aviso.get("_id"));
93              }
94          }
95          return nuevos;
96      }
```

Figura 18. *Método registrar(evento): inserción individual y tratamiento de DuplicateKeyException.*

Índice, consulta e integración con RabbitMQ

Agregué el índice compuesto clabe_fecha, con CLABE ascendente y fecha descendente, y programé ultimos(clabe) para devolver hasta diez avisos ordenados del más reciente al más antiguo. En el consumidor de RabbitMQ cambié solamente la llamada principal por bitacora.registrar(evento); los avisos dejaron de escribirse en la bandeja anterior de PostgreSQL.

```
138     /** Se corre una vez al arrancar. Crear un indice que ya existe igual no hace nada. */
139
140     @EventListener(ApplicationReadyEvent.class)
141     public void crearIndice() {
142         String nombre = mongo.indexOps(COLECCION).ensureIndex(
143             new Index().on("clabe", Sort.Direction.ASC).on("fecha", Sort.Direction.DESC).named("clabe_fecha"));
144         log.info("[avisos] indice {} listo en {}", nombre, COLECCION);
145     }
146
147     /** Los ULTIMOS avisos de una CLABE, del mas reciente al mas viejo, tal como estan guardados. */
148
149     public List<Document> ultimos(String clabe) {
150         Query consulta = Query.query(Criteria.where("clabe").is(clabe))
151             .with(Sort.by(Sort.Direction.DESC, "fecha"))
152             .limit(ULTIMOS);
153         return mongo.find(consulta, Document.class, COLECCION);
154     }
```

Figura 19. *Creación del índice clabe_fecha y método ultimos(clabe).*

```
78          try {
79              nuevos = bitacora.registrar(evento);
80          } catch (RuntimeException fallaNuestra) {
```

Figura 20. *ConsumidorDeTransferencias.java registra los avisos mediante BitacoraDeAvisos.*

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs notificacion | grep avisos
notificacion-1  | 2026-11-12T04:11:10.078Z  INFO 1 --- [notificacion] [           main] m.m.n.avisos.BitacoraDeAvisos           : [avisos] indice clabe_fecha listo en avisos
```

Figura 21. *Log del servicio notificacion que confirma la creación del índice clabe_fecha.*

Las tres formas y los últimos diez

Realicé una transferencia de $200 de Ana a Beto y otra de $200 de Ana a Carla; las dos respondieron 201 Created y COMPLETADA. Después consulté MongoDB y encontré cuatro documentos: dos SMS para Ana, un CORREO para Beto y un PUSH para Carla. El filtro por la existencia del campo telefono devolvió solamente los dos avisos SMS.



Figura 22. *Transferencias de prueba a Beto y Carla con respuesta 201 Created.*

```
[
  {
    _id: 'c8e04e4f-8701-4d3f-bb5c-229323c6618d:002180000000000001',
    eventoId: 'c8e04e4f-8701-4d3f-bb5c-229323c6618d',
    transferenciaId: Long('13'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('200'),
    saldo: Decimal128('580.00'),
    fecha: ISODate('2026-11-12T04:16:14.921Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 200, saldo resultante 580.00'
  },
  {
    _id: 'c8e04e4f-8701-4d3f-bb5c-229323c6618d:002180000000000002',
    eventoId: 'c8e04e4f-8701-4d3f-bb5c-229323c6618d',
    transferenciaId: Long('13'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('200'),
    saldo: Decimal128('920.00'),
    fecha: ISODate('2026-11-12T04:16:14.921Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'TRANSFERENCIA_RECIBIDA',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 200, saldo resultante 920.00'
  },
```

Figura 23. *Primeros documentos guardados: SMS de Ana y CORREO de Beto.*

```
  ,,
  {
    _id: '0666ce5b-31ad-4025-9ad6-85751a5053b2:002180000000000001',
    eventoId: '0666ce5b-31ad-4025-9ad6-85751a5053b2',
    transferenciaId: Long('14'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('200'),
    saldo: Decimal128('380.00'),
    fecha: ISODate('2026-11-12T04:16:42.279Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 200, saldo resultante 380.00'
  },
  {
    _id: '0666ce5b-31ad-4025-9ad6-85751a5053b2:002180000000000003',
    eventoId: '0666ce5b-31ad-4025-9ad6-85751a5053b2',
    transferenciaId: Long('14'),
    clabe: '002180000000000003',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('200'),
    saldo: Decimal128('500.00'),
    fecha: ISODate('2026-11-12T04:16:42.279Z'),
    canal: 'PUSH',
    token: 'fcm:demo-002180000000000003',
    titulo: 'TRANSFERENCIA_RECIBIDA',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 200, saldo resultante 500.00'
  }
]
```

Figura 24. *Otros documentos guardados: SMS de Ana y PUSH de Carla.*

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({telefono: {$exists: true}}, {clabe: 1, canal: 1, telefono: 1})'
[
  {
    _id: 'c8e04e4f-8701-4d3f-bb5c-229323c6618d:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '0666ce5b-31ad-4025-9ad6-85751a5053b2:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
```

Figura 25. *Consulta por el campo telefono: únicamente los dos avisos SMS de Ana.*

Luego ejecuté doce transferencias adicionales de $1 de Ana a Beto. Con el aviso que ya tenía, Beto acumuló trece documentos en MongoDB. Sin embargo, el endpoint /api/notificaciones/002180000000000002 devolvió exactamente diez, todos de tipo CORREO, comenzando con la transferencia 26 y terminando con la 17. Eso comprobó el límite y el orden descendente.

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % for i in $(seq 1 12); do curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1}'; done
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % curl -s http://localhost:8080/api/notificaciones/002180000000000002
[{"_id":"9d2e6662-ddfd-42a3-939b-be9563164340:0021800000000000002","eventoId":"9d2e6662-ddfd-42a3-939b-be9563164340","transferenciaId":26,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","mon
to":1,"saldo":932.00,"fecha":"2026-11-12T04:27:29.789+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 932.00"
},{"_id":"c01c0d1e-2f94-41f5-b45f-8fe075476303:0021800000000000002","eventoId":"c01c0d1e-2f94-41f5-b45f-8fe075476303","transferenciaId":25,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","mo
nto":1,"saldo":931.00,"fecha":"2026-11-12T04:27:29.663+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 931.00
"},{"_id":"d7de92c2-0296-41b5-b000-5bbb1ee49cd4:0021800000000000002","eventoId":"d7de92c2-0296-41b5-b000-5bbb1ee49cd4","transferenciaId":24,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","m
onto":1,"saldo":930.00,"fecha":"2026-11-12T04:27:29.496+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 930.0
0"},{"_id":"5390828b-b772-4ff2-ba9c-f56856666a908:0021800000000000002","eventoId":"5390828b-b772-4ff2-ba9c-f56856666a908","transferenciaId":23,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","
monto":1,"saldo":929.00,"fecha":"2026-11-12T04:27:29.273+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 929.
00"},{"_id":"929d5626-a07e-470a-9592-0b27e607a76f:0021800000000000002","eventoId":"929d5626-a07e-470a-9592-0b27e607a76f","transferenciaId":22,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA",
"monto":1,"saldo":928.00,"fecha":"2026-11-12T04:27:29.003+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 928
.00"},{"_id":"11efc2b4-79cd-4b33-9e0b-a3462c746d48:0021800000000000002","eventoId":"11efc2b4-79cd-4b33-9e0b-a3462c746d48","transferenciaId":21,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA"
,"monto":1,"saldo":927.00,"fecha":"2026-11-12T04:27:27.260+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 92
7.00"},{"_id":"39f58994-4409-490e-b602-b3ffcb3e3857:0021800000000000002","eventoId":"39f58994-4409-490e-b602-b3ffcb3e3857","transferenciaId":20,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA
","monto":1,"saldo":926.00,"fecha":"2026-11-12T04:27:26.864+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 9
26.00"},{"_id":"c62c8c56-ea36-4cd8-93fb-31981f881a8e:0021800000000000002","eventoId":"c62c8c56-ea36-4cd8-93fb-31981f881a8e","transferenciaId":19,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBID
A","monto":1,"saldo":925.00,"fecha":"2026-11-12T04:27:26.504+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante
925.00"},{"_id":"6a907172-c0f1-4601-8250-302879de6f50:0021800000000000002","eventoId":"6a907172-c0f1-4601-8250-302879de6f50","transferenciaId":18,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBI
DA","monto":1,"saldo":924.00,"fecha":"2026-11-12T04:27:26.183+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante
 924.00"},{"_id":"0c9dbd1c-f8a9-47eb-94c1-b4a1a3af55e9:0021800000000000002","eventoId":"0c9dbd1c-f8a9-47eb-94c1-b4a1a3af55e9","transferenciaId":17,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIB
IDA","monto":1,"saldo":923.00,"fecha":"2026-11-12T04:27:25.945+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"TRANSFERENCIA_RECIBIDA","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultant
e 923.00"}]%
```

Figura 26. *Doce transferencias de $1 y respuesta del endpoint con los últimos diez avisos de Beto.*

Comparación del índice con explain()

Comprobé en MongoDB que existieran el índice automático _id_ y el compuesto clabe_fecha. Después comparé la misma consulta con y sin ese índice mediante explain("executionStats"). Con IXSCAN MongoDB examinó diez documentos; al forzar el recorrido natural de la colección con hint({$natural: 1}), utilizó COLLSCAN y examinó los 28 documentos almacenados.

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.getIndexes()'
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { clabe: 1, fecha: -1 }, name: 'clabe_fecha' }
]
```

Figura 27. *Índices de la colección avisos: _id_ y clabe_fecha.*

```
printJson({plan:e.queryPlanner.winningPlan, documentosExaminados:e.executionStats.totalDoc
'
{
  plan: {
    isCached: false,
    stage: 'LIMIT',
    limitAmount: 10,
    inputStage: {
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: {
          clabe: 1,
          fecha: -1
        },
        indexName: 'clabe_fecha',
        isMultiKey: false,
        multiKeyPaths: {
          clabe: [],
          fecha: []
        },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          clabe: [
            '["002180000000000002", "002180000000000002"]'
          ],
          fecha: [
            '[MaxKey, MinKey]'
          ]
        }
      }
    }
  },
  documentosExaminados: 10,
  devueltos: 10
}
```

Figura 28. *Consulta con índice: plan LIMIT, FETCH, IXSCAN y diez documentos examinados.*

```
printjson({plan:e.queryPlanner.winningPlan, documentosExaminados:e.executionStats.totalD
'
{
  plan: {
    isCached: false,
    stage: 'SORT',
    sortPattern: {
      fecha: -1
    },
    memLimit: 104857600,
    limitAmount: 10,
    type: 'simple',
    inputStage: {
      stage: 'COLLSCAN',
      filter: {
        clabe: {
          '$eq': '002180000000000002'
        }
      },
      direction: 'forward'
    }
  },
  documentosExaminados: 28,
  devueltos: 10
}
```

Figura 29. *Consulta sin índice: plan SORT, COLLSCAN y 28 documentos examinados.*

Consulta	Etapas	Documentos examinados	Devueltos
Con índice	LIMIT / FETCH / IXSCAN	10	10
Sin índice	SORT / COLLSCAN	28	10

Ambas consultas devolvieron los mismos diez avisos, pero el índice redujo el trabajo de lectura de 28 a 10 documentos. La ejecución con índice tardó 9 ms y la otra 8 ms, una diferencia mínima en una colección pequeña; aquí la ventaja demostrable fue el número de documentos examinados, no una mejora garantizada de tiempo en cada ejecución.

5. El evento duplicado

Para comprobar la idempotencia publiqué dos veces el mismo evento en el exchange transferencias mediante la API de RabbitMQ. Utilicé las CLABEs de prueba terminadas en 091 y 092 para no mezclarlo con las cuentas normales. Las dos publicaciones respondieron routed:true, pero MongoDB conservó solamente dos avisos asociados al eventoId fijo.

Los logs de notificacion mostraron que los dos _id ya existían y que el evento repetido no se duplicó. Aunque RabbitMQ entrega mensajes al menos una vez y una reentrega puede ocurrir, el identificador único de cada aviso evitó guardar cuatro documentos.

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % curl -u mexibanco:mexibanco -X POST http://localhost:15672/api/exchanges/%2F/transferencias/publish \
  -H 'Content-Type: application/json' \
  -d '{"properties":{"delivery_mode":2,"content_type":"application/json"},"routing_key":"transferencia.realizada","payload_encoding":"string","payload":"{\"eventoId\":\"11111111-2222-4333-8444-555555555
555\",\"tipo\":\"TransferenciaRealizada\",\"transferenciaId\":9001,\"clabeOrigen\":\"002180000000000091\",\"clabeDestino\":\"002180000000000092\",\"monto\":25,\"saldoOrigen\":475,\"saldoDestino\":525,\"
fecha\":\"2026-10-31T13:00:00Z\"}"}'
{"routed":true}%
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % curl -u mexibanco:mexibanco -X POST http://localhost:15672/api/exchanges/%2F/transferencias/publish \
  -H 'Content-Type: application/json' \
  -d '{"properties":{"delivery_mode":2,"content_type":"application/json"},"routing_key":"transferencia.realizada","payload_encoding":"string","payload":"{\"eventoId\":\"11111111-2222-4333-8444-555555555
555\",\"tipo\":\"TransferenciaRealizada\",\"transferenciaId\":9001,\"clabeOrigen\":\"002180000000000091\",\"clabeDestino\":\"002180000000000092\",\"monto\":25,\"saldoOrigen\":475,\"saldoDestino\":525,\"
fecha\":\"2026-10-31T13:00:00Z\"}"}'
{"routed":true}%
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments({eventoId: "11111111-2222-4333-8444-555555555555"})'
2
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose logs notificacion | grep "ya estaba"
notificacion-1  | 2026-11-12T04:39:45.829Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos           : [avisos] 11111111-2222-4333-8444-555555555555:002180000000000091 ya est
aba: evento repetido, no se duplica
notificacion-1  | 2026-11-12T04:39:45.835Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos           : [avisos] 11111111-2222-4333-8444-555555555555:002180000000000092 ya est
aba: evento repetido, no se duplica
```

Figura 30. *Dos publicaciones del mismo evento, solo dos avisos y mensajes ya estaba en los logs.*

6. El dato viejo

Redis muestra una copia, PostgreSQL autoriza

Primero consulté el saldo de Ana y anoté los $368.00 que tenía antes de la prueba. Después cambié el saldo directamente en PostgreSQL a $5.00 con un UPDATE, sin pasar por CuentaService, por lo que @CacheEvict no pudo invalidar automáticamente el valor que estaba en Redis.

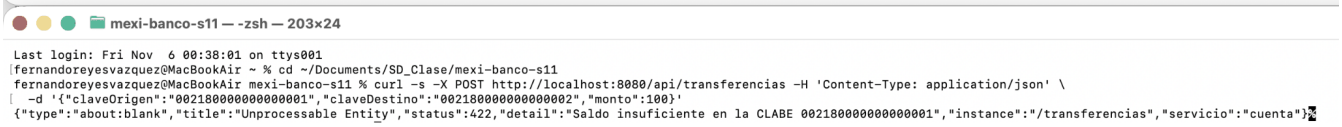
```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
{"clabe":"002180000000000001","titular":"Ana","saldo":368.00,"leidaEn":"2026-11-12T04:46:32.345494916Z"}
```

Figura 31. *Saldo inicial de Ana: $368.00 antes de la modificación directa.*

Ejecuté un ciclo para revisar el TTL y el saldo cada cinco segundos. Mientras Redis indicaba 14, 9 y 3 segundos restantes, el GET todavía mostraba $368.00, aunque PostgreSQL ya tenía $5.00. Cuando TTL devolvió -2, la entrada había caducado y las consultas comenzaron a devolver $5.00 desde la base de datos.

Durante esa ventana intenté transferir $100 a Beto desde otra Terminal. La API rechazó la operación con estado 422 y el mensaje Saldo insuficiente. Esto demostró que la autorización de cargos sigue consultando el saldo real en PostgreSQL y no se deja llevar por una copia desactualizada de Redis.

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % curl -s http://localhost:8080/api/cuentas/002180000000000001
{"clabe":"002180000000000001","titular":"Ana","saldo":368.00,"leidaEn":"2026-11-12T04:56:06.336079210Z"}%
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T db psql -U mexibanco -d cuenta -c "update cuenta set saldo = 5 where clabe = '002180000000000001'"
UPDATE 1
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % for i in $(seq 1 8); do
  docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
  curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
  sleep 5
done
14
{"clabe":"002180000000000001","titular":"Ana","saldo":368.00,"leidaEn":"2026-11-12T04:56:06.336079210Z"}
9
{"clabe":"002180000000000001","titular":"Ana","saldo":368.00,"leidaEn":"2026-11-12T04:56:06.336079210Z"}
3
{"clabe":"002180000000000001","titular":"Ana","saldo":368.00,"leidaEn":"2026-11-12T04:56:06.336079210Z"}
-2
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-12T04:56:38.744684211Z"}
25
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-12T04:56:38.744684211Z"}
19
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-12T04:56:38.744684211Z"}
14
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-12T04:56:38.744684211Z"}
8
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-12T04:56:38.744684211Z"}
```




```
Last login: Fri Nov  6 00:38:01 on ttys001
fernandoreyesvazquez@MacBookAir ~ % cd ~/Documents/SD_Clase/mexi-banco-s11
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":100}'
{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Saldo insuficiente en la CLABE 002180000000000001","instance":"/transferencias","servicio":"cuenta"}%
```

Figura 32. *TTL y saldos distintos durante la prueba, junto con el rechazo 422 de la transferencia.*

Restauración de los datos

Al finalizar restauré los $368.00 de Ana directamente en PostgreSQL. El DEL de Redis devolvió 0 porque la clave ya había caducado, y una nueva consulta por la API confirmó el saldo original. Así dejé las cuentas como estaban antes de provocar el fallo.

```
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T db psql -U mexibanco -d cuenta -c "update cuenta set saldo = 368 where clabe = '002180000000000001'"
UPDATE 1
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % docker compose exec -T redis redis-cli DEL cuentas::002180000000000001
0
fernandoreyesvazquez@MacBookAir mexi-banco-s11 % curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
{"clabe":"002180000000000001","titular":"Ana","saldo":368.00,"leidaEn":"2026-11-12T05:00:30.619481096Z"}
```

Figura 33. *Restauración del saldo de Ana a $368.00 y verificación final por la API.*

7. Preguntas de cierre

¿Por qué MongoDB para los avisos y no para el saldo?

Los avisos no tienen siempre la misma estructura: uno puede traer teléfono, otro correo y otro token de push. Además, se guardan como historial y se consultan por cuenta y fecha, por lo que MongoDB se adapta bien a esa información. El saldo, en cambio, necesita movimientos y actualizaciones consistentes para no crear ni perder dinero; por eso PostgreSQL sigue siendo la fuente de verdad.

¿Qué hit ratio medí en A y B, y qué compré con la caché?

En A obtuve 99 % (99 hits y un miss), porque repetí consultas sin cambiar saldos. En B obtuve 83.33 % (100 hits y 20 misses), porque las transferencias obligaban a invalidar las copias. Mis promedios con caché fueron 14.37 y 15.87 ms; sin ella, 27.36 y 16.68 ms. Lo más evidente fue la capacidad: bajé de cien lecturas a PostgreSQL a una sola por corrida de cien consultas.

¿Qué pasaría si el cargo se autorizara con el saldo de Redis?

Si el cargo se autorizara con el saldo de Redis, podrían realizarse transferencias utilizando información desactualizada. Durante la prueba, Redis todavía mostraba $368 cuando PostgreSQL ya tenía $5. Si el sistema hubiera confiado en esa copia, habría podido autorizar una transferencia de $100 sin fondos suficientes, generando inconsistencias en los saldos. Sin embargo, la transferencia fue rechazada con un error 422 porque la autorización utilizó el saldo real de PostgreSQL. Esto demuestra que Redis sirve para acelerar las consultas, pero no debe utilizarse para autorizar movimientos de dinero.

¿Por qué DEL y no SET, y por qué después del commit?

Al hacer DEL, la próxima lectura vuelve a PostgreSQL y recupera el saldo ya confirmado; con un SET podríamos escribir una copia equivocada si hubo otras operaciones concurrentes. Además, invalidar después del commit evita borrar la caché por una transacción que termine revirtiéndose y evita que otra consulta vuelva a guardar un saldo anterior antes de que se confirme el movimiento.